X1 Capstone 1: refactor a messy pipeline

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track X: Capstones **2**
X1 Capstone 1: refactor a messy pipeline 3
X1-L1 Capstone 1: refactor the NAT pipeline 3
Track X best-practice checklist 9
Appendix: solutions and answer keys **10**

Track X: Capstones

Three projects that combine the tracks, each checked against tests or a hidden key.

X1 Capstone 1: refactor a messy pipeline

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Refactor a notebook-style pipeline to the standard, with tests on a local SparkSession.

X1-L1 Capstone 1: refactor the NAT pipeline

Concept

This capstone brings Track A together. You start from `practice/capstone/before_pipeline.py`: a notebook exported as one script, the way many pipelines begin. It runs, and it breaks almost every rule in `CLAUDE.md`. You finish with `practice/exercises/x1_pipeline.py`, refactored to the standard and passing 11 acceptance tests on a local SparkSession.

What the starting code gets wrong (each one is a lesson you have done):

- A SparkSession created at import time, and a hard-coded column map (A1).
- Bronze written with `mode("overwrite")`, and counted before the write (A2).
- No missing-column check before renames, and casts that hide bad values (A2, D3).
- A MERGE with no deduplication of the source (A3).
- A gold `groupBy` that drops `tracker_id`, a `sum(when(...))` that gives null, and an unguarded division (A5, D4).
- Checks stored as text in a mutable default list, with repeated `count()` calls (A6, D3).
- A run log with no schema, `uuid` and `datetime` never imported, and `utcnow()` (A7, D1).
- A secret placeholder in code (A10).

What you deliver: ten functions with fixed names and signatures, listed in the skeleton. The acceptance tests in `practice/tests/test_x1_pipeline.py` import them. They check behaviour, not style, so run ruff and mypy as well.

How to work. Read the brief and the tests first. Implement one function at a time, in the order of the pipeline: config, bronze, silver, gold, checks, run log. Run the tests after each function. Compare with `practice/solutions/x1_pipeline.py` only once your tests pass.

The exercises below practise the pure-Python parts in the browser. The Spark parts run on your laptop. Setup is in `practice/README.md`.

Why it matters

Refactoring a working pipeline is harder than writing a new one. The old code produces numbers people rely on, and every change risks moving them. Acceptance tests with known answers let you change structure without changing results. This is how you will improve real pipelines at work: one tested function at a time, never a rewrite in one go.

Worked example: weak code

```
1 def run(path, wave_id, checks=[]):
2     df = spark.read.option("header", True).csv(path)
3     n = df.count()
4     df.write.mode("overwrite").saveAsTable("nat_raw")
```

1. **Line 1.** One function does everything, with no types and a mutable default.
2. **Line 4.** Overwrite on bronze, after a separate count that scans the source again.

Worked example: strong code

```
1 def ingest_bronze(
2     spark: SparkSession, df: DataFrame, table: str, source_file: str,
      delivery_version: int
3 ) -> int:
4     """Append a delivery to bronze. Bronze is append-only."""
5     audited = add_audit_columns(df, source_file, delivery_version,
      str(uuid.uuid4()))
6     audited.write.format("delta").mode("append").saveAsTable(table)
7     return last_write_rows(spark, table)
```

1. **Line 6.** Append with audit columns.
2. **Line 7.** Rows written come from the Delta history, not a second scan.

Refactor one function at a time, in pipeline order, with acceptance tests running after every change.

Recap card

- **Capstone 1 workflow** Read and run the acceptance tests, then refactor one function at a time in pipeline order.
- **Where the capstone files live** Start: practice/capstone/before_pipeline.py. Yours: practice/exercises/x1_pipeline.py. Tests: practice/tests/test_x1_pipeline.py.
- **When to open the reference solution** Only after your tests pass. Then compare structure, not just results.

Practice

Exercise 1. Multiple choice (level 1)

Before you change a line of the starting pipeline, what do you do first?

- a) Read the acceptance tests and run them, so you know what 'done' means
- b) Rewrite the whole script in one go
- c) Run ruff --fix on it
- d) Delete the defects you recognise

Why?

- r1) Tests define the target behaviour. Running them first shows where you start and proves each later step.
- r2) Tests are always written after refactoring.

Exercise 2. Spot the bug (level 2)

These lines come from the starting pipeline. Tap the line that destroys earlier deliveries.

```
1 df = spark.read.option("header", True).csv(path)
2 n = df.count()
3 df.write.mode("overwrite").saveAsTable("nat_raw")
4 for src, tgt in COLUMN_MAP.items():
5     df = df.withColumnRenamed(src, tgt)
```

Write down the line number of the bug.

Exercise 3. Put in order (level 2)

Order the refactor so each step can be tested before the next depends on it.

- ingest_bronze with audit columns
- run_checks and enforce
- transform_silver and upsert_silver
- load_config with validation
- build_run_record and log_run
- compute_gold

Exercise 4. Write the function (level 3)

Write `validate_config(raw: dict[str, object]) -> dict[str, object]`, the pure-Python core of `load_config`. Required keys: `tracker_id`, `column_map`, `type_map`, `merge_keys`, `weight_sum_min`, `weight_sum_max`, `max_null_weights`. Raise `ValueError` naming every missing key, in that order. Also raise

`ValueError` if `merge_keys` is empty or `weight_sum_min` exceeds `weight_sum_max`. Return `raw` unchanged when valid.

Your code must pass these checks:

- Return the config unchanged when it is valid.
- Name every missing key in one error, so the config is fixed in one go.
- Empty merge keys and min above max are invalid.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Module constant.
REQUIRED_CONFIG_KEYS = ...

# Check a pipeline config before any step uses it.
def validate_config(
    raw: dict[str, object],
) -> dict[str, object]: ...
```

Starter code:

```python
def validate_config(raw):
    return raw
```

Exercise 5. Write the function (level 4)

Write `enforce(results: list[dict[str, str]]) -> None`. Each result has `check_name`, `status` (`"PASS"`, `"FAIL"` or `"WARN"`) and `severity` (`"CRITICAL"` or `"WARN"`). Raise `RuntimeError` naming every failed CRITICAL check, in order. WARN results and passes never raise.

Your code must pass these checks:

- WARN results log and continue. They must not raise.
- Name every failed CRITICAL check in one error, and only those.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Stop the pipeline when a CRITICAL check failed.
def enforce(results: list[dict[str, str]]) -> None: ...
```

Starter code:

```python
def enforce(results):
    for r in results:
        if r["status"] != "PASS":
            raise RuntimeError(r["check_name"])
```

Exercise 6. Refactor (level 5)

Refactor the starting pipeline's run log record into `build_run_record(pipeline_name: str, tracker_id: str, wave_id: str, layer: str, rows_processed: int, status: str, error_message: str | None = None) -> dict[str, object]`. Return the keys `run_id`, `pipeline_name`, `tracker_id`, `wave_id`, `layer`, `rows_processed`, `status`, `error_message` and `run_timestamp`. `run_id` is a new UUID string. `run_timestamp` is an aware UTC datetime.

Your code must pass these checks:

- Return exactly the nine run log keys.
- A new UUID string per call, and an aware UTC datetime, not a string.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# One run log record with a new run id and an aware UTC timestamp.
def build_run_record(
    pipeline_name: str,
    tracker_id: str,
    wave_id: str,
    layer: str,
    rows_processed: int,
    status: str,
    error_message: str | None = None,
) -> dict[str, object]: ...
```

Starter code:

```python
def build_run_record(pipeline_name, tracker_id, wave_id, layer,
    rows_processed, status, error_message: str = None):
     return {"run_id": str(uuid.uuid4()), "wave_id": wave_id, "rows":
    rows_processed,
            "status": status, "error_message": error_message,
            "ts": datetime.utcnow().isoformat()}
```

Practical: Capstone 1: refactor the NAT pipeline on your laptop

Platform: Laptop. You need Java 17 or 21 and the `spark` dependency group (see practice/README.md). The acceptance tests run on a local SparkSession with Delta Lake.

1. Read practice/capstone/before_pipeline.py and list every defect you see. Compare your list with the concept section.
2. Read practice/tests/test_x1_pipeline.py. Write down, in one line each, what the 11 tests require.
3. Run `pytest practice/tests/test_x1_pipeline.py`. Every test fails with NotImplementedError. That is your starting point.

4. Implement load_config and PipelineConfig.**post_init**. Run the two config tests until they pass.
5. Implement ingest_bronze, then transform_silver and upsert_silver, then compute_gold. Run the tests after each function.
6. Implement run_checks and enforce, then build_run_record, log_run and get_unprocessed_waves.
7. Run `ruff check practice` and `mypy`. Fix every finding.
8. When all 11 tests pass, compare your module with practice/solutions/x1_pipeline.py. Note two things the reference does differently and why.

Sources

- The reference solution passes all 11 acceptance tests, and each of five deliberate defects (overwrite on bronze, missing tracker in the gold grain, unfiltered silver waves, cast instead of try_cast, no source dedup before MERGE) fails at least one test. (verified: Local run on PySpark 4.0.4 and delta-spark 4.0.1, September 2026.)

Track X best-practice checklist

☐ Read the brief and restate the goal before you touch code.

☐ Work to the CLAUDE.md standard, not to the starting code.

☐ Run the acceptance tests before and after each change.

☐ Record every Copilot or assistant output you rely on, with how you checked it.

☐ Compare your work with the reference or key only after you submit.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

X1-L1 Capstone 1: refactor the NAT pipeline

X1-L1 exercise 1

Hints. (1) How will you know you are done? (2) The acceptance tests. (3) Read and run them first.

Answer. a) Read the acceptance tests and run them, so you know what 'done' means Because: r1) Tests define the target behaviour. Running them first shows where you start and proves each later step.

Tests first, then one function at a time.

Common wrong answers.

- `b`: A rewrite in one go changes everything at once, so you cannot tell which change broke a number.
- `c`: Formatting helps later. It does not tell you what the code must do.
- `d`: Deleting code without tests risks removing behaviour someone relies on.
- `a|r2`: Right answer, wrong reason. Tests come first, so they can check each step.

X1-L1 exercise 2

Hints. (1) Which line changes a table? (2) Bronze is append-only. (3) The write mode.

Answer. The bug is on line 3.

`mode("overwrite")` replaces every earlier delivery in bronze. Use `mode("append")` with audit columns.

Common wrong answers.

- Line 2: The extra count costs a scan, but it does not destroy data. The write below it does.
- Line 5: A silent rename is a problem, but earlier deliveries survive it.

X1-L1 exercise 3

Hints. (1) What does every step need? (2) Follow the data: bronze, silver, gold. (3) Checks and logging wrap the finished layers.

Answer.

1. load_config with validation
2. ingest_bronze with audit columns
3. transform_silver and upsert_silver
4. compute_gold
5. run_checks and enforce
6. build_run_record and log_run

Config, then the layers in data order, then checks and the run log.

Common wrong answers.

- Every later step reads the config. Build and validate it first.
- Gold is computed from silver. Silver comes first.

X1-L1 exercise 4

Hints. (1) Collect all missing keys before raising. (2) Keep the required keys in a constant. (3) Then check the two value rules.

Answer.

```python
REQUIRED_CONFIG_KEYS = (
    "tracker_id",
    "column_map",
    "type_map",
    "merge_keys",
    "weight_sum_min",
    "weight_sum_max",
    "max_null_weights",
)


def validate_config(raw: dict[str, object]) -> dict[str, object]:
    """Check a pipeline config before any step uses it.

    Args:
        raw: Parsed JSON config.

    Returns:
        The same config, if valid.

    Raises:
        ValueError: If keys are missing or values are impossible.
    """
    missing = [key for key in REQUIRED_CONFIG_KEYS if key not in raw]
```

```
    if missing:
        raise ValueError(f"config is missing keys: {missing}")
    if not raw["merge_keys"]:
        raise ValueError("merge_keys must not be empty")
    if float(raw["weight_sum_min"]) > float(raw["weight_sum_max"]):  # type: ignore[arg-type]
        raise ValueError("weight_sum_min exceeds weight_sum_max")
    return raw
```

Fail fast, and fail once with every problem listed. A config fixed one key per run wastes a morning.

X1-L1 exercise 5

Hints. (1) Filter on severity and status together. (2) Collect names, then raise once. (3) WARN never raises.

Answer.

```
def enforce(results: list[dict[str, str]]) -> None:
    """Stop the pipeline when a CRITICAL check failed.

    Args:
        results: Structured check results.

    Raises:
        RuntimeError: Naming every failed CRITICAL check.
    """
    failed = [
        r["check_name"] for r in results if r["severity"] == "CRITICAL" and r["status"] == "FAIL"
    ]
    if failed:
        raise RuntimeError(f"CRITICAL checks failed: {failed}")
```

CRITICAL halts, WARN logs and continues: the severity rule from your standard, in five lines.

X1-L1 exercise 6

Hints. (1) Import uuid and datetime, timezone. (2) error_message: str | None = None. (3) datetime.now(timezone.utc), kept as a datetime.

Answer.

```
import uuid
from datetime import datetime, timezone


def build_run_record(
    pipeline_name: str,
    tracker_id: str,
    wave_id: str,
    layer: str,
```

```
    rows_processed: int,
    status: str,
    error_message: str | None = None,
) -> dict[str, object]:
    """One run log record with a new run id and an aware UTC timestamp.

    Args:
        pipeline_name: Job name, such as nat_silver_pipeline.
        tracker_id: Tracker processed.
        wave_id: Wave processed.
        layer: bronze, silver or gold.
        rows_processed: Rows written by the run.
        status: Outcome of the run.
        error_message: Error text for a failed run.

    Returns:
        The record, ready for a schema-typed append.
    """
    return {
        "run_id": str(uuid.uuid4()),
        "pipeline_name": pipeline_name,
        "tracker_id": tracker_id,
        "wave_id": wave_id,
        "layer": layer,
        "rows_processed": rows_processed,
        "status": status,
        "error_message": error_message,
        "run_timestamp": datetime.now(timezone.utc),
    }
```

Three defects fixed in one function: missing imports, a lying type hint and a deprecated, naive timestamp.